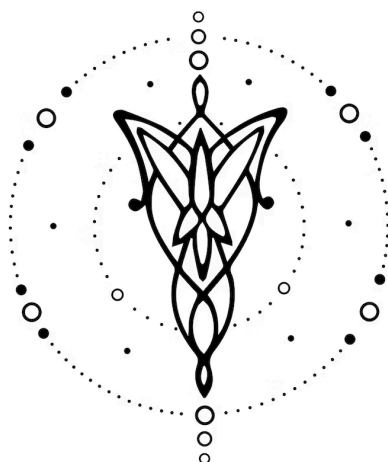




Compilador de JSON a SQL

DISEÑO E IMPLEMENTACIÓN

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Martín Alejandro	Barnatán	64463	mbarnatan@itba.edu.ar
Celestino	Garrós	64375	cgarros@itba.edu.ar
Ignacio	Pedemonte Berthoud	64908	ipedemonteberthoud@itba.edu.ar
Leo	Weitz	64365	lweitz@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en el siguiente [repositorio](#).

3. Dominio

Desarrollar un lenguaje que permita diseñar consultas de SQL con todos sus modificadores a partir de un formato JSON. El lenguaje debe permitir realizar consultas simples, así como definir queries como variables y “nestearlas” de esta manera con otras, posibilitando así escribir consultas más complejas.

Para reducir la complejidad del proyecto, nos limitamos dentro del mundo de acciones de PostgreSQL a las consultas, sin incluir otras como inserciones, modificaciones o eliminaciones.

La implementación satisfactoria de este lenguaje permitirá ofrecer una alternativa para realizar consultas SQL que facilite la sintaxis en diversos casos. Por ejemplo, en aquellos en que se desea “nestar” múltiples consultas SQL, donde en muchos sistemas de base de datos esta sintaxis suele ser muy engorrosa, como por ejemplo en PostgreSQL.

4. Construcciones

El lenguaje desarrollado debe ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán crear una o varias consultas SQL como objetos JSON, cada una conteniendo obligatoriamente los campos “attributes” y “from”, así como opcionalmente “where”, “group by” y “order by”.
- (II). Los atributos “from”, “attributes”, “order by” y “group by” tendrán como valor un array de strings.
- (III). El atributo “where” tendrá como valor un objeto JSON, dentro del cual se especificará la/las condiciones que correspondan, teniendo la posibilidad de utilizar los atributos “and” u “or”.
- (IV). Dentro de las consultas se podrán utilizar las funciones de agregación, tales como “count()” o “avg()”.
- (V). Dentro de una consulta se podrán crear otras consultas auxiliares con un atributo “name”, siendo estas accesibles sólo dentro de su consulta padre.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que genere una consulta SQL simple con un “select” y un “from” usando una sola tabla.
- (II). Un programa que genere una consulta SQL usando el atributo “where”.
- (III). Un programa que genere una consulta SQL usando el atributo “group by”.
- (IV). Un programa que genere una consulta SQL usando el atributo “order by”.
- (V). Un programa que genere una consulta SQL combinando los atributos “where” y “group by”.
- (VI). Un programa que genere una consulta SQL utilizando alguna función de agregación.

- (VII). Un programa que genere una consulta SQL utilizando otra consulta auxiliar en el atributo “from”.
- (VIII). Un programa que genere una consulta SQL simple con un “select”, un “from” con múltiples tablas y un “where”.
- (IX). Un programa que genere una consulta SQL con varias condiciones dentro de los atributos “and” y “or”.
- (X). Un programa que genere una consulta SQL combinando todos los atributos ofrecidos.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa mal formado (no en formato JSON).
- (II). Un programa que tenga 2 atributos duplicados en una misma consulta.
- (III). Un programa que utilice una variable inexistente.
- (IV). Un programa que no contenga los atributos obligatorios (“attributes” y “from”).
- (V). Un programa que utilice una función de agregación inválida.

6. Ejemplos

Consulta SQL simple con un “select” y un “from” usando una sola tabla::

```
{
  "from": ["table_1"],
  "attributes": ["attr_1", "attr_2", ..., "attr_n"]
}
```

Consulta SQL con otra “nesteada” y una condición “where” que usa el operador =, “not null” y un “and”:

```
{
  "auxiliary": [
    {
      "name": "auxi",
      "from": ["table_2"],
      "attributes": ["count(distinct attr_1)"]
    }
  ],
  "from": ["table_1"],
  "where": {
    "operator": "=",
    "first_value": "attr_1",
    "second_value": "auxi",
    "and": [
      {
        "attribute": "attr_2",
        "null": "no"
      }
    ]
  }
}
```

```
}  
}
```